
Westward Trail: Developing and Scrutinizing a Local LLM Game Master via Prompt Engineering

Sander De Bleecker (Student number 852656007) ¹

Abstract

I developed an Oregon-Trail-style survival game *Westward Trail*, in which a locally served open-weight LLM acts as game master: it narrates events and attaches numeric consequences to the player's choices. The program itself still holds the authoritative game state and validates every LLM-proposed effect against hard rules, so the game can never enter an invalid state *and* every turn doubles as a labeled test case. On top of this app I run controlled scrutiny experiments comparing three strategies along four metrics: JSON format reliability, hard-rule adherence, state tracking over long conversations (with and without ground-truth state in the prompt), and recall of facts stated once at game start. Format compliance turns out to be nearly solved even by a minimal prompt (99.5% parse rate), but rule adherence is not: a single worked example cuts hard-rule violations from 185 to 14 per 100 turns, while writing the rules out verbatim helps not at all, because the dominant error is a schema misreading (absolute values echoed into a delta field) that examples fix and rules cannot. Without ground-truth state in the prompt, the model's believed food count drifts by over 100 lbs within 20 turns, and recall of facts stated once collapses from 100% to 6% at the moment the game intro slides out of the rolling context window; the model was never remembering, it was reading.

1. Introduction

The first idea was to create a roguelike survival game heavily inspired by Oregon Trail, in which a family tries to survive a multi-month journey with a cart through an unforgiving wasteland. There is a dark side to the game where it often just goes wrong for the company and you have to replay it multiple times until luck is on your side.

This concept was appealing to me, as the chances must have been predefined to have certain events occur during the playthrough. Now, with how easy it is to run LLMs locally,

the chances can be made nondeterministic or unknown by simply using the weights of an LLM. At first all chances were done by the LLM itself; later on the game had to be playable, so some chances were left to the program. The two mechanisms, prompted event results versus purely stochastic event results, are cleanly separated by giving the pure stochastic chances their own mechanic, namely "scavenging": a mechanic where the player can choose to scavenge for items at night, which has a chance of finding items but also a chance of getting hurt or losing items.

2. Methods

2.1. Game loop

The architecture was laid out around how the few game actions you can take were supposed to be handled.

A day and its decision: every turn in the game is a day, which starts with a description of how the day is going and with 3 options for how it should be further planned out. The game then waits for the user's decision. *The end of day and its consequences:* with the context of the chosen option, the LLM makes up a JSON object of rewards or penalties the travelling party gets. *The optional scavenging:* on the left, during the user's decision, the user can choose to scavenge for items at night. This is a stochastic mechanic with real stakes.

Here, the whole day including consequences used to be generated by one prompt; this changed as the accuracy was not up to par. Later on, the day was split into multiple prompts (two for the narration, to make it more story-like by switching prompt), while the consequences were split into a separate prompt giving complete context so the prompt could focus on the consequences. Furthermore, a check is done to ensure the consequences are valid.

You can find the prompts in the prompts folder of the GitHub repository; their usage can be found in `prompts.py`. An HTML website is loaded as the interface for the user via `serve.py` and `index.html`.

Asynchronously, while the user is still deciding, the LLM is already generating a scene image for the day; this is done in the background and does not block the user from making a

decision.

2.2. Inference engine and model choice

For the LLM, everything was run locally on Ollama at first. Due to project requirements, the same model used in Ollama was then run via PyTorch.

The preferred model for the game loop is the instruction-tuned Gemma E4B checkpoint (Gemma Team et al., 2024). Beforehand, different variations of Qwen and Mistral were tested. Gemma E4B was chosen as it seemed more manipulable via prompt, and it was more efficient on the NVIDIA GeForce RTX 5080 due to being NF4-quantized. Trial and error found that Qwen2 often needed more retries for outputting valid JSON, that Qwen3 models were an improvement but heavier, and that Mistral had more hallucinations.

2.3. Gradual improvement via prompt engineering

The app did not arrive at its current shape in one pass: the development journal (`journey.md`) records sixteen small stages, and nearly every one of them is a prompt intervention rather than an algorithmic change. They group into four recurring lessons.

Pick a model the prompt can steer. The very first obstacle was the model, not the prompt: a reasoning-tuned Qwen3 checkpoint intermingled its chain-of-thought with the expected JSON, and disabling reasoning did not fully stop the leakage, which led to a curated catalog of non-thinking models as the single source of truth for model selection. The default context limit also proved too short to keep games interesting beyond ten turns, so the context window was raised explicitly and treated as a budget from then on (the rolling history window of Section 2.4 follows from this).

Give every prompt one job. The largest quality jumps came from decomposition. When one call produced both the options and their rewards, the numbers often contradicted the text (saving an ox by resting would *cost* an ox), which motivated the narrator/scorer split of Section 2.5. The same principle was applied twice more: encounter days with strangers got their own prompt, separate from ordinary travel days, so the two event kinds stop blending into a monotone middle ground; and a dedicated post-decision prompt narrates how the chosen option played out; initially the consequences were visible on the choice itself, which was convenient for debugging but ruined the immersion of finding out how the day went.

Prompts are code and not like code. Midway through, all prompts were moved out of Python strings into plain text files, which made them readable, diffable, and cheap to iterate on; one such iteration uncovered a hand-made bug where the worked JSON example inside a prompt was itself invalid JSON: the model had been imitating a broken template. A

Table 1. Prompt strategies. Each adds one ingredient.

Strategy	Adds over the previous
<code>minimal</code>	Task description + bare JSON schema
<code>fewshot_schema</code>	+ one fully worked example turn
<code>rules_explicit</code>	+ the hard game rules written out verbatim

later pass rewrote the surface tone of every prompt to be consistently western-flavored, an improvement that would have been tedious while prompts lived inline. The subtlest bug was context leakage: the fixed 2-bullet cost of night scavenging, present in the prompt’s state summary, kept surfacing in generated stories as bullets the party “found”; the fix was to remove state the prompt did not need, i.e. prompt *minimization* rather than addition.

Know what not to prompt. Playtesting showed the game could be won by simply forcing maximum effort every day, and no phrasing made the model punish this reliably. The risk of serious disease when pushing too hard was therefore moved into the harness as a hard-coded stochastic rule the model only narrates, the origin of the prompted-versus-stochastic separation described in the introduction. Where the model repeatedly failed at a job, the improvement was to take the job away from it.

There are of course more, smaller but noteworthy actions, such as setting a tone for the output by defining that tone in the prompt, giving an example of the output, and stripping whole concepts from a prompt when they sidetrack it from its main objective.

2.4. Prompt strategies

We compare three strategies that each vary exactly one dimension over the previous, so effects attribute cleanly (Table 1). All three request the same JSON object: a `believed_state` (the model’s own reconstruction of the current state, used for the state-tracking experiment), a 2–3 sentence `narrative`, and exactly three `options`, each with an `effects` dict of numeric deltas.

2.5. Splitting narration from scoring

At first the app revealed contradictions between the text of the option and the consequence paired to it. For example, a “brief rest” costing an ox, or +105 health for an unhurt character. The program therefore splits each turn into a *narrator* call (temperature 0.8) and a *scorer* call (temperature 0.1, given the true state, the narrative, and the three option texts, and asked only for effects under the hard rules plus explicit coherence constraints). A third call narrates the outcome after the game has applied the chosen effects.

Table 2. Hard-rule violation taxonomy audited by engine.check_rules.

Class	Meaning
miles_exceeds_max	option moves > 25 miles (or < 0)
overdraw:res	food/oxen/bullets/money pushed below 0
hunt.*	hunt with < 10 bullets, without the -10 bullet cost, or yielding > 100 lbs
healing_exceeds_max	one event heals > 40 HP
unknown_party_member	effects name a nonexistent person
recovery_without_health	sick flag cleared at $\leq$ 30 HP
status_on_dead_member	flags set on a 0 HP member
non_numeric/non_boolean	wrong value types
unknown_sentiment	sentiment outside the 6-word scale

A `--single-call` flag restores the one-call behavior for A/B comparison. The scripted experiments deliberately use the one-call strategies of Table 1, as they scrutinize the program’s behavior.

3. Experimental Setup

There is no external dataset: the game itself is the test set, and the harness state is the ground truth (a deliberate consequence of the assignment’s local, reproducible setup). A scripted bot plays full games of up to 20 turns for every combination of strategy $\times$ mode (*guided/blind*) $\times$ seed, choosing uniformly among the offered options. Every turn is appended as one JSON line with a deduplication key, so runs are resumable and a Ctrl-C never corrupts data; completed runs are skipped on rerun.

Per turn we log: parse success and the raw output on failure (format reliability); the violation list from `check_rules`, giving a violation taxonomy for free (rule adherence, Table 2); and per-field `|believed - true|` for day, miles, food, bullets, and money (state tracking). In guided mode the true state is shown every turn (can the model even copy correctly?); in blind mode it is shown only on turn 1 and the model must reconstruct it from the event history. For the memory experiment, three party facts stated once in the fixed game intro (Elsie’s weak ankle, Jonas’s former trade, Ruben being the best shot) are quizzed at turns 5/10/15/20 at temperature 0, scored by whether the gold answer appears in the reply. `analyze.py` turns the log into the tables and figures below.

4. Results

4.1. Format reliability and rule adherence

Table 3 reports parse rate and violation density per strategy. (Turn counts differ because games end early on death or arrival; runs lasted 12–20 turns.)

Format reliability is nearly solved even for `minimal`: 196 of 197 turns parsed, the tolerant parser absorb-

Table 3. Format reliability and hard-rule violation rate per prompt strategy.

Strategy	Turns	Parse rate	Viol. / 100 turns
minimal	66	0.985	184.6
fewshot_schema	74	1.000	13.5
rules_explicit	57	1.000	198.2

Table 4. Mean absolute error of the model’s believed state vs. ground truth (parsed turns only).

Strategy	Mode	day	miles	food	bullets	money
minimal	guided	0.0	0.0	0.2	0.0	0.0
minimal	blind	0.8	42.9	126.9	6.2	35.5
fewshot	guided	0.0	0.5	0.6	0.0	0.0
fewshot	blind	0.8	22.8	56.4	3.2	36.2
rules	guided	0.0	0.0	0.0	0.0	0.0
rules	blind	0.8	28.4	129.0	4.8	37.2

ing markdown fences and stray prose (the lone failure was an *unevaluated arithmetic expression*, `"miles": 200 - (5 + 10 + ...)`, foreshadowing the arithmetic weakness below). Rule adherence is not solved, and the ordering is non-monotonic: `fewshot_schema` commits 14 $\times$ fewer violations than `minimal`, yet `rules_explicit` is no better. The taxonomy explains it: nearly all `minimal` and `rules_explicit` violations are `healing_exceeds_max`: the model echoing the party’s *absolute* health values into the *delta* field. One worked example with a negative delta (`"Jonas": -10`) eliminates the class; stating the rule verbatim does not, because the model does not know its numbers are read as deltas. A *schema-interpretation* error is fixed by an example, not a rule. As in the app, small models act on declarative facts far more reliably than on conditionals they must evaluate themselves.

4.2. State tracking and drift

Guided error is ~ 0 everywhere (Table 4): the model copies a shown state faultlessly, so drift is not a transcription problem. Blind, belief diverges immediately (Figure 1): food is worst (MAE up to 129), miles reaches 43, yet the day counter stays near-exact (0.8). The model tracks the calendar but loses any quantity built from many small arithmetic updates, the failure the *unevaluated-expression* parse error made explicit; the worked example halves food drift (56.4).

One drift source was our own bug: food consumption (5 lbs per living member per day) was originally charged silently by the harness, so blind-mode food drifted by the daily meal cost regardless of recall; we were measuring a hidden rule, not context fidelity. Moving the rule into the prompt removed the confound: a state-tracking experiment is only

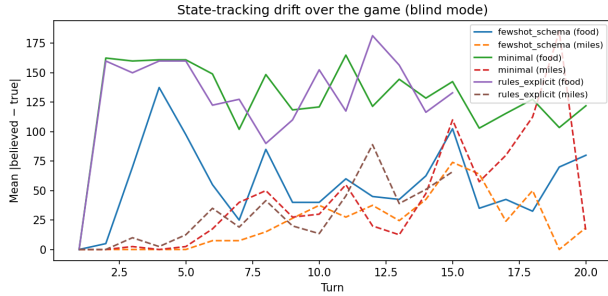


Figure 1. State-tracking drift over the game in blind mode: mean $|\text{believed} - \text{true}|$ per turn for food (solid) and miles (dashed), per strategy.

Table 5. Recall of party facts stated once at game start, quizzed at turns 5/10/15/20 (blind mode).

Strategy	Quiz answers	Recall accuracy
minimal	32	0.500
fewshot_schema	32	0.656
rules_explicit	20	0.850

valid if the state’s dynamics are observable to the model.

4.3. Memory of facts stated once

Recall does not decay gradually; it falls off a cliff (Figure 2): 24/24 answers correct at turns 5 and 10, then 25% at turn 15 and 6% at turn 20, exactly when the game intro (the facts’ only source) leaves the 6,000-character history window. The model was never *remembering*, only *reading*. This also confounds the per-strategy scores in Table 5: *rules_explicit* looks best (0.850) only because its games ended before the post-ejection quizzes. The lesson: durable facts must be re-injected into every prompt, not left behind in the transcript.

4.4. Qualitative findings from the app

Three further errors surfaced in play, each with a fix. (1) *Narrative–arithmetic incoherence*: one call doing both story and numbers produced consequences unrelated to its prose, cured by the narrator/scorer split (Section 2.5). (2) *Option monotony*: a schema example mentioning “push on / wait / spend” taught the model to offer that trio every day, cured by demanding options differ *in kind* plus scheduled encounter days. (3) *Context leakage*: the 2-bullet scavenge cost surfaced in stories as “the party found 2 bullets”, cured by trimming redundant state from the prompt. A cap on sentiment swings was reverted: it only fired when the model had good reason to swing, manufacturing violations rather than catching them.

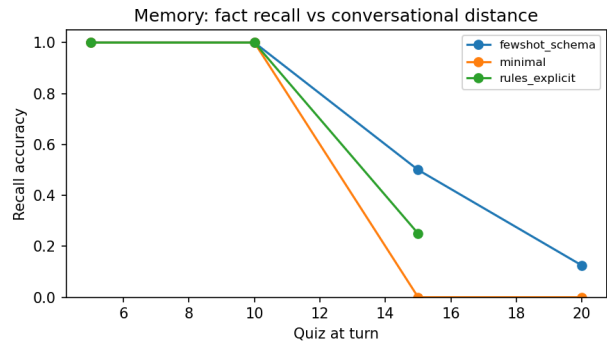


Figure 2. Fact recall vs. the turn at which the quiz was asked.

5. Future Work

Note that further optimizations could be done by either using VLLM or a smaller model where fit. One area which has not been explored is running multiple models in parallel, where one model is used for the narration and another for the scoring. One reason for this is that the behaviour of the inference engine plays a critical role in the performance when running multiple models in parallel. The gameplay could be made more interesting by having an orchestrator prompt decide which event prompt to take and the event prompt being variable in itself, so that the model can be steered to a certain type of event. I would end with noting how it is very phantomable given this experiment to image games containing LLMs as game masters or mere characters coming to the industry in the near future.

References

Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., et al. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*, 2024.